

CPPDescent

4bf8c4d (with uncommitted changes)

Generated by Doxygen 1.9.7

Chapter 1

CPP-Descent

Part of the “Software Development for Computing Systems” course of the Department of Informatics and Telecommunications, UoA for the first semester of the academic year 2023-2024.

A library that implements the KNN-Graph creation and search algorithms described in [1], [2].

1.1 Contributors

- **Konstantinos Chousos** (1115202000215, sdi2000215 at di.uoa.gr)
 - 1st submission
 - * Build/Testing/Code coverage pipeline
 - * ADTVector, ADTList, ADTPQueue
 - * Brute Force K-NN Graph creation. NN-Descent Graph creation
 - * (Incomplete) Query point
 - 2nd submission
 - * I/O of graphs in binary files
 - * Recall function
 - * Metric functions
 - * Query point
 - * Early termination
 - * Local join
- **Anastasios-Fedon Seitanidis** (1115202000179, sdi2000179 at di.uoa.gr)
 - 1st submission
 - * ADTMap, ADTGraph
 - * Brute Force K-NN Graph creation
 - * NN-Descent Graph creation
 - 2nd submission
 - * Graph refactoring for more efficient neighbors retrieval

1.2 Dependencies

For local development, you will need `Make` and `CMake`. To install them on an apt-based linux distro, run the following commands:

```
$ sudo apt update && sudo apt upgrade
$ sudo apt install make
$ sudo apt install cmake
```

1.2.1 Minor dependencies

- The project uses [Google Test](#) for its testing needs. Gtest is used as a git submodule of this repo, that is downloaded by CMake the first time you clone. That means that nothing extra needs to be installed to run the tests.
- If you would like to generate the documentation, you will need `doxygen` installed.

1.3 Running the project

To run the main executable, the following commands suffice:

```
$ cmake -S . -B build
$ cmake --build build
$ ./build/app/app <K> <filepath-to-dataset> <dimensions> <1 or 2>
```

The last argument concerns the metric function that will be used. 1 means that the euclidean distance will be used and 2 stands for the manhattan distance.

1.3.1 Results

The *recall* percentage that is printed during execution is computed by the following formula, derived from [1, Sec. 3.2].

The recall of one object is the number of its true K-NN members found divided by K. The recall of an approximate K-NN is the average recall of all objects.

1.4 Documentation

1.4.1 Code Structure

In the [wiki](#), you can find documentation generated by [Doxygen](#). It is also available in [pdf form](#).

1.4.2 Code Coverage

This project uses [LCOV](#) for its code coverage needs. The results are uploaded upon commit [here](#).

1.4.3 Profiling

`Callgrind` and `Qcachegrind` are used for any profiling purposes.

1.5 Bibliography

[1] W. Dong, C. Moses, and K. Li, “Efficient k-nearest neighbor graph construction for generic similarity measures,” in Proceedings of the 20th international conference on World wide web, in WWW '11. New York, NY, USA: Association for Computing Machinery, Mar. 2011, pp. 577–586. doi: 10.1145/1963405.1963487.

[2] “How PyNNDescent works — pynndescent 0.5.0 documentation.” Accessed: Oct. 10, 2023. [Online]. Available: https://pynndescent.readthedocs.io/en/latest/how_pynndescent_works.html

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

cppdescent	Functions for the creation of a K-NN graph	??
----------------------------	--	----

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Graph	..	??
GraphVertex	..	??
GraphVertexPair	..	??
List		
ADT List	..	??
ListNode		
Node of a list	..	??
Map	..	??
MapNode	..	??
PQueue		
ADT Priority Queue	..	??
Vector		
ADT Vector	..	??
vectorNode		
Class for the vector Node object	..	??

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

ADTGraph.hpp		
Abstract undirected graph with weighted edges	...	??
ADTList.hpp	...	??
ADTMap.hpp	...	??
ADTPQueue.hpp	...	??
ADTVector.hpp		
ADTVector implementation using a dynamic array	...	??
common.hpp	...	??
cppdescent.hpp	...	??
ADTGraph.cpp	...	??
ADTList.cpp		
Implementation of an ADT linked list	...	??
ADTMap.cpp		
Implementation of ADTMap	...	??
ADTPQueue.cpp		
An ADT Priority Queue implemented using a heap	...	??
ADTVector.cpp		
Implementation of ADTVector using a dynamic array	...	??
cppdescent.cpp		
Implementation of the cppdescent library	...	??

Chapter 5

Namespace Documentation

5.1 cppdescent Namespace Reference

Functions for the creation of a K-NN graph.

Functions

- void **deleteFloat** (Pointer value)
- float * **createFloat** (float value)
- float **compareFloats** (Pointer a, Pointer b)
- int **deleteDatapointVectors** ([Vector](#) *vec)
- int **compareGraphVertices** (Pointer vertex1, Pointer vertex2)
- int **compareVertices** (Pointer first, Pointer second)
- [Vector](#) * **readBinData** (const char *fp, int dimensions)
Reads the data from a binary file.
- void **writeBinGraph** (const char *fp, [Graph](#) *graph, int K)
Writes a computed graph to a binary file.
- [Graph](#) * **readBinGraph** (const char *fp, int dimensions, DistanceFunc distance)
Reads a graph from a binary file. To work correctly, the file needs to be first created from the 'writeBinGraph' function.
- float **recall** ([Graph](#) *bfGraph, [Graph](#) *nnGraph, int N, int K)
Returns the recall of the graph computed by NN-Descent, compared to the brute force graph.
- float **euclideanDistance** (Pointer a, Pointer b)
Returns the Euclidean distance between two points of arbitrary dimension.
- float **manhattanDistance** (Pointer a, Pointer b)
Returns the Manhattan distance between two points of arbitrary dimension.
- int **compareEdgesEuclidean** (Pointer first, Pointer second)
Compare edges using the euclideanDistance function.
- int **compareVertexPairsEuclidean** (Pointer first, Pointer second)
- int **compareEdgesManhattan** (Pointer first, Pointer second)
Compare edges using the manhattanDistance function.
- [Graph](#) * **KNNBruteForceGraph** ([Vector](#) *data, int K, CompareFunc compare, DistanceFunc distance)
Computes the K-NN graph using brute force.
- [Graph](#) * **NNDescent_KNNGraph** ([Vector](#) *data, int K, float delta, DistanceFunc distance)
Computes the K-NN graph for the given dataset using the NN-Descent algorithm.
- [PQueue](#) * **NNDescent_Query** ([Graph](#) *graph, int K, CompareFunc compare, [Vector](#) *query)
Computes the K Nearest Neighbors of the query point in the graph.

5.1.1 Detailed Description

Functions for the creation of a K-NN graph.

5.1.2 Function Documentation

5.1.2.1 `compareEdgesEuclidean()`

```
int cppdescent::compareEdgesEuclidean (
    Pointer first,
    Pointer second )
```

Compare edges using the `euclideanDistance` function.

Parameters

<i>first</i>	A Pointer to the first element.
<i>second</i>	A Pointer to the second element.

Returns

int

5.1.2.2 `compareEdgesManhattan()`

```
int cppdescent::compareEdgesManhattan (
    Pointer first,
    Pointer second )
```

Compare edges using the `manhattanDistance` function.

Parameters

<i>first</i>	A Pointer to the first element.
<i>second</i>	A Pointer to the second element.

Returns

int

5.1.2.3 `euclideanDistance()`

```
float cppdescent::euclideanDistance (
    Pointer a,
    Pointer b )
```

Returns the Euclidean distance between two points of arbitrary dimension.

Parameters

<i>first</i>	A pointer to the first point.
<i>second</i>	A pointer to the second point.

Returns

long double The Euclidean distance.

5.1.2.4 KNNBruteForceGraph()

```
Graph * cppdescent::KNNBruteForceGraph (
    Vector * data,
    int K,
    CompareFunc compare,
    DistanceFunc distance )
```

Computes the K-NN graph using brute force.

Useful for testing the recall and the performance of our NN-Descent algorithm.

The user is responsible deletion of the allocated memory of the graph.

Parameters

<i>data</i>	A pointer to the parent N-sized vector.
<i>K</i>	The number of Nearest Neighbors to find.
<i>compare</i>	The function to use to compare the distances.
<i>distance</i>	The function that computes the distance between two points (vectors).

Returns

Graph* A pointer to the optimal K-NN graph.

5.1.2.5 manhattanDistance()

```
float cppdescent::manhattanDistance (
    Pointer a,
    Pointer b )
```

Returns the Manhattan distance between two points of arbitrary dimension.

Parameters

<i>first</i>	A pointer to the first point.
<i>second</i>	A pointer to the second point.

Returns

long double The Manhattan distance.

5.1.2.6 NNDescent_KNNGraph()

```
Graph * cppdescent::NNDescent_KNNGraph (
    Vector * data,
    int K,
    float delta,
    DistanceFunc distance )
```

Computes the K-NN graph for the given dataset using the NN-Descent algorithm.

The user is responsible deletion of the allocated memory of the returned graph.

Parameters

<i>data</i>	A Vector of the vertices of the graph.
<i>K</i>	The number of nearest neighbors to compute.
<i>delta</i>	The iterations will stop when the number of edges that were updated is less than $\text{delta} * N * K$.
<i>distance</i>	The function to be used to compute the distances between vertices.

Returns

Graph* The complete K-NN graph of the dataset.

5.1.2.7 NNDescent_Query()

```
PQueue * cppdescent::NNDescent_Query (
    Graph * graph,
    int K,
    CompareFunc compare,
    Vector * query )
```

Computes the K Nearest Neighbors of the query point in the graph.

Parameters

<i>graph</i>	The K-NN Graph , in which we search for neighbors of the query.
<i>K</i>	
<i>compare</i>	The function used to compare edges.
<i>query</i>	The query point, given as a Vector . Must be of the same dimensions as the rest points of the graph.

Returns

PQueue* A priority queue of the K-NN of the query, with 'max' being the most distant point relative to the query. It contains GraphVertexPairs from the query point directed to the other nodes. If the query point is of wrong dimensions, then nullptr is returned.

5.1.2.8 readBinData()

```
Vector * cppdescent::readBinData (
    const char * fp,
    int dimensions )
```

Reads the data from a binary file.

The data read are stored in a N-sized [Vector](#), where N is the <uint32_t> at the start of the file. Each node of this vector is a pointer to another 100-sized vector, of which the values are pointers to floats.

All of the vectors (the parent N-sized and each of the 100-sized) must be freed by the user.

Parameters

<i>fp</i>	A string (char *) of the filepath to the binary dataset.
<i>dimensions</i>	The dimension of each point.

Returns

Vector* An N-sized vector with vectors for each element.

5.1.2.9 readBinGraph()

```
Graph * cppdescent::readBinGraph (
    const char * fp,
    int dimensions,
    DistanceFunc distance )
```

Reads a graph from a binary file. To work correctly, the file needs to be first created from the 'writeBinGraph' function.

Parameters

<i>fp</i>	The filepath.
<i>dimensions</i>	The dimensions of the datapoints.
<i>distance</i>	A function to compute the distance between the vertices.

Returns

Graph*

5.1.2.10 recall()

```
float cppdescent::recall (
    Graph * bfGraph,
    Graph * nnGraph,
    int N,
    int K )
```

Returns the recall of the graph computed by NN-Descent, compared to the brute force graph.

Parameters

<i>bfGraph</i>	
<i>nnGraph</i>	
<i>N</i>	
<i>K</i>	

Returns

float

5.1.2.11 writeBinGraph()

```
void cppdescent::writeBinGraph (
    const char * fp,
    Graph * graph,
    int K )
```

Writes a computed graph to a binary file.

The structure of the binary file that will be created is the following:

1. <uint32_t> *N*: the number of the vertices
2. *N* * 100 floats: each vertex of 100 dimensions
3. *N* * *K* * 2 int: pairs of ints that describe the edges between the vertices. Each int represents the index of the vertex in the order above. This will be handy when we reconstruct the graph using the `getAt()` function of the vector.

Parameters

<i>fp</i>	The filepath to the created file.
<i>K</i>	
<i>graph</i>	

Chapter 6

Class Documentation

6.1 Graph Class Reference

Public Member Functions

- **Graph** (CompareFunc compare, DestroyFunc destroy, DestroyFunc vecDestroy=NULLPTR)
- int **getSize** ()
- void **insertVertex** (Pointer vertex)
Insert a vertex to the graph (if it doesn't already exist).
- **List** * **getVertices** ()
The returned list needs to be deleted, but the list's elements are pointers to the vector's elements, so the list shouldn't have a destroyValue.
- void **removeVertex** (Pointer vertex)
- void **insertEdge** (Pointer vertex1, Pointer vertex2, float weight)
- void **removeEdge** (Pointer vertex1, Pointer vertex2)
- float **getWeight** (Pointer vertex1, Pointer vertex2)
- **List** * **getAdjacent** (Pointer vertex)
- **List** * **getAdjacentVertices** (Pointer vertex)
- **PQueue** * **getAdjacentPQ** (Pointer vertex)
- **List** * **getReverseAdjacent** (Pointer vertex)
- **List** * **getReverseAdjacentVertices** (Pointer vertex)
- **PQueue** * **getReverseAdjacentPQ** (Pointer vertex)
- **List** * **getGeneralNeighbors** (Pointer vertex)
- **List** * **getGeneralNeighborsVertices** (Pointer vertex)
- **PQueue** * **getGeneralNeighborsPQ** (Pointer vertex)
- bool **isNeighbor** (Pointer v1, Pointer v2)
- void **setHashFunction** (HashFunc hash)
- CompareFunc **getCompareData** ()
- CompareFunc **getCompareVertices** ()
- DestroyFunc **getDestroy** ()
- DestroyFunc **getDestroyData** ()
- HashFunc **getHash** ()
- **Vector** * **getVec** ()
- **Map** * **getMap** ()

6.1.1 Member Function Documentation

6.1.1.1 getVertices()

```
List * Graph::getVertices ( )
```

The returned list needs to be deleted, but the list's elements are pointers the the vector's elements, so the list shouldn't have a `destroyValue`.

Returns

List*

6.1.1.2 insertVertex()

```
void Graph::insertVertex (
    Pointer vertex )
```

Insert a vertex to the graph (if it doesn't already exist).

Parameters

<i>vertex</i>	A Pointer to the vertex to be unserted.
---------------	---

The documentation for this class was generated from the following files:

- [ADTGraph.hpp](#)
- [ADTGraph.cpp](#)

6.2 GraphVertex Class Reference

Public Member Functions

- **GraphVertex** (Pointer data, [Graph](#) *owner)
- void **addNeighbor** (Pointer neighbor)
- void **addReverse** (Pointer reverse)
- void **removeNeighbor** (Pointer neighbor, CompareFunc compare)
- void **removeReverse** (Pointer reverse, CompareFunc compare)
- [PQueue](#) * **getNeighbors** ()
- [PQueue](#) * **getReverse** ()
- Pointer **getData** ()
- [Graph](#) * **getOwner** ()
- void **check** ()
- bool **checked** ()

The documentation for this class was generated from the following files:

- [ADTGraph.hpp](#)
- [ADTGraph.cpp](#)

6.3 GraphVertexPair Class Reference

Public Member Functions

- **GraphVertexPair** ([Graph](#) *owner, Pointer vertex1, Pointer vertex2)
- Pointer **getVertex1** ()
- Pointer **getVertex2** ()
- [Graph](#) * **getOwner** ()

The documentation for this class was generated from the following file:

- [ADTGraph.hpp](#)

6.4 List Class Reference

ADT [List](#).

```
#include <ADTList.hpp>
```

Public Member Functions

- [List](#) (DestroyFunc value=NULLPTR)
Construct a new [List](#) object.
- [~List](#) ()
Destroy the [List](#) object.
- void [insertNext](#) ([ListNode](#) *node, Pointer value)
Adds a node after the given node.
- void [removeNext](#) ([ListNode](#) *node)
Removes the node after the given node.
- Pointer [find](#) (Pointer value, CompareFunc compare)
Finds the first value equal to the value parameter.
- DestroyFunc [setDestroyValue](#) (DestroyFunc value)
Set the Destroy Value object to a new value.
- int [getSize](#) ()
Get the size of the [List](#).
- int [increaseSize](#) ()
Increase the list size.
- int [decreaseSize](#) ()
Decrease the list size.
- [ListNode](#) * [getHead](#) ()
Getter for the head of the [List](#).
- [ListNode](#) * [getTail](#) ()
Getter for the tail of the [List](#).
- [ListNode](#) * [next](#) ([ListNode](#) *node)
Returns the node after the given one.
- Pointer [nodeValue](#) ([ListNode](#) *node)
Returns the value of the node.
- [ListNode](#) * [findNode](#) (Pointer value, CompareFunc compare)
Finds the first node that has value equal to the value parameter.
- int [mergeLists](#) ([List](#) *list)
Concatenation of 2 lists (Result this->list).

6.4.1 Detailed Description

ADT [List](#).

An Abstract Data Type [List](#) with linear access to data, and addition and removal to/from arbitrary positions.

6.4.2 Constructor & Destructor Documentation

6.4.2.1 List()

```
List::List (
    DestroyFunc value = nullptr )
```

Construct a new [List](#) object.

Parameters

<i>destroyValue</i>	If destroyValue != nullptr, the destroyValue function will be called each time a node is removed.
---------------------	---

head is a virtual node that is always present in the list.

Parameters

<i>destroyValue</i>	The destroy function for the values of the list. Defaults to nullptr.
---------------------	---

6.4.2.2 ~List()

```
List::~~List ( )
```

Destroy the [List](#) object.

If the destroyValue function is not nullptr, it is called for every value.

6.4.3 Member Function Documentation

6.4.3.1 decreaseSize()

```
int List::decreaseSize ( )
```

Decrease the list size.

Returns

int The updated size.

6.4.3.2 find()

```
Pointer List::find (
    Pointer value,
    CompareFunc compare )
```

Finds the first value equal to the value parameter.

Find the first value equal to the value parameter.

Parameters

<i>value</i>	The value based on which the search is done.
<i>compare</i>	A pointer to the function that will compare the node values.

Returns

Pointer A pointer to the resulting node.

Parameters

<i>value</i>	The value to search for.
<i>compare</i>	A pointer to the compare function.

Returns

Pointer A pointer to found value.

6.4.3.3 findNode()

```
ListNode * List::findNode (
    Pointer value,
    CompareFunc compare )
```

Finds the first node that has value equal to the value parameter.

Find the first node with value equal to the value parameter.

Parameters

<i>value</i>	The value to search.
<i>compare</i>	The compare function to use.

Returns

ListNode* A pointer to the resulting node.

Parameters

<i>value</i>	The value to search for.
<i>compare</i>	A pointer to the compare function.

Returns

ListNode* A pointer to the found node.

6.4.3.4 getHead()

```
ListNode * List::getHead ( )
```

Getter for the head of the [List](#).

Get the first element of the [List](#).

Returns

ListNode* The head.

Returns

ListNode* Pointer to the first element.

6.4.3.5 getSize()

```
int List::getSize ( )
```

Get the size of the [List](#).

Get the size of the list.

The size is updated upon entry/removal of nodes, so the retrieval of the size is O(1).

Returns

int

Returns

int The size.

6.4.3.6 getTail()

```
ListNode * List::getTail ( )
```

Getter for the tail of the [List](#).

Get the last element of the list.

Returns

ListNode* The tail.

If the list is empty, the virtual head is the tail, so we return nullptr.

Returns

ListNode* Pointer to the last element.

6.4.3.7 increaseSize()

```
int List::increaseSize ( )
```

Increase the list size.

Returns

int The updated size.

6.4.3.8 insertNext()

```
void List::insertNext (
    ListNode * node,
    Pointer value )
```

Adds a node *after* the given node.

Insert a new element after the node.

Parameters

<i>node</i>	The node that the new node will be placed after.
<i>value</i>	The value of the new node.

Parameters

<i>node</i>	The node after which the element is inserted.
<i>value</i>	The value of the new element.

6.4.3.9 mergeLists()

```
int List::mergeLists (
    List * list )
```

Concatenation of 2 lists (Result this->list).

Concatenation of 2 lists (Result: list1->list2).

Parameters

<i>list</i>	List to be merged with this.
-------------	------------------------------

Returns

int Returns 0 in case of success, -1 in any other case.

Parameters

<i>list</i>	List to be merged with this.
-------------	--

Returns

int Returns 0 in case of success, -1 in any other case.

6.4.3.10 next()

```
ListNode * List::next (
    ListNode * node )
```

Returns the node after the given one.

Get the next node.

Parameters

<i>node</i>	
-------------	--

Returns

ListNode* A pointer to the next node.

Parameters

<i>node</i>	
-------------	--

Returns

ListNode*

6.4.3.11 nodeValue()

```
Pointer List::nodeValue (
    ListNode * node )
```

Returns the value of the node.

Get the node's value.

Parameters

<i>node</i>	
-------------	--

Returns

Pointer A generic pointer to the value.

Parameters

<i>node</i>	
-------------	--

Returns

Pointer

6.4.3.12 removeNext()

```
void List::removeNext (
    ListNode * node )
```

Removes the node *after* the given node.

Removes the node after the given node.

Parameters

<i>node</i>	The node before the node to be removed.
-------------	---

Parameters

<i>node</i>	The node after which the element is removed.
-------------	--

6.4.3.13 setDestroyValue()

```
DestroyFunc List::setDestroyValue (
    DestroyFunc value )
```

Set the Destroy Value object to a new value.

Set the destroy value function.

Parameters

<i>value</i>	A pointer to the new destroy funciton.
--------------	--

Returns

DestroyFunc A pointer to the old destroy function.

Parameters

<i>value</i>	The new destroy function.
--------------	---------------------------

Returns

DestroyFunc The old destroy function.

The documentation for this class was generated from the following files:

- ADTList.hpp
- [ADTList.cpp](#)

6.5 ListNode Class Reference

Node of a list.

```
#include <ADTList.hpp>
```

Public Member Functions

- [ListNode](#) ()
Construct a new [List](#) Node object that is empty.
- [ListNode](#) (Pointer value)
Construct a new [List](#) Node object with value 'value'.
- void [setNext](#) ([ListNode](#) *next)
Setter for the next node.
- [ListNode](#) * [getNext](#) ()
Getter for the next node.
- Pointer [getValue](#) ()
Getter for the value of the node.

6.5.1 Detailed Description

Node of a list.

A list node is represented by its value. It also holds a pointer to the next node in the list.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 [ListNode](#)() [1/2]

```
ListNode::ListNode ( ) [inline]
```

Construct a new [List](#) Node object that is empty.

6.5.2.2 [ListNode](#)() [2/2]

```
ListNode::ListNode (
    Pointer value ) [inline]
```

Construct a new [List](#) Node object with value 'value'.

Parameters

<i>value</i>	a generic pointer to the value of the node.
--------------	---

6.5.3 Member Function Documentation

6.5.3.1 getNext()

```
ListNode * ListNode::getNext ( )
```

Getter for the next node.

Get the next node of the list.

Returns

ListNode* The pointer to the next node.

Returns

ListNode* The next node.

6.5.3.2 getValue()

```
Pointer ListNode::getValue ( )
```

Getter for the value of the node.

Get the value of the current node.

Returns

Pointer A generic pointer to the value.

Returns

Pointer The value of the current node.

6.5.3.3 setNext()

```
void ListNode::setNext (
    ListNode * next )
```

Setter for the next node.

Set the given node as the next one.

Parameters

<i>next</i>	The pointer to the next node.
-------------	-------------------------------

Parameters

<i>next</i>	The node to be next.
-------------	----------------------

The documentation for this class was generated from the following files:

- ADTList.hpp
- [ADTList.cpp](#)

6.6 Map Class Reference

Public Member Functions

- [Map](#) (CompareFunc compare, DestroyFunc destroyKey, DestroyFunc destroyValue)
Construct a new [Map::Map](#) object.
- [~Map](#) ()
Destroy the [Map::Map](#) object.
- void [insert](#) (Pointer key, Pointer value)
Add a new pair (key --> value) in the map.
- bool [remove](#) (Pointer key)
Remove from the map the pair (key --> value) that corresponds to the given key.
- Pointer [find](#) (Pointer key)
- void [rehash](#) ()
Extend the hash table in case the load factor exceeds the predefined threshold.
- void [setCapacity](#) (int capacity)
- void [setDestroyKey](#) (DestroyFunc destroyKey)
- void [setDestroyValue](#) (DestroyFunc destroyValue)
- void [setHashFunction](#) (HashFunc hash)
- [MapNode](#) ** [getArray](#) ()
- int [getCapacity](#) ()
- int [getSize](#) ()
- int [getDeleted](#) ()
- DestroyFunc [getDestroyKey](#) ()
- DestroyFunc [getDestroyValue](#) ()
- HashFunc [getHashFunction](#) ()
- [MapNode](#) * [getFirst](#) ()
- [MapNode](#) * [getNext](#) ([MapNode](#) *node)
- [MapNode](#) * [findNode](#) (Pointer key)

6.6.1 Constructor & Destructor Documentation

6.6.1.1 Map()

```
Map::Map (
    CompareFunc compare,
    DestroyFunc destroyKey,
    DestroyFunc destroyValue )
```

Construct a new [Map::Map](#) object.

Parameters

<i>compare</i>	A compare function to keep the map sorted
<i>destroyKey</i>	A destroy function for the key of every node
<i>destroyValue</i>	A destroy function fro the balue of every node

6.6.1.2 ~Map()

```
Map::~~Map ( )
```

Destroy the [Map::Map](#) object.

6.6.2 Member Function Documentation**6.6.2.1 findNode()**

```
MapNode * Map::findNode (
    Pointer key )
```

Parameters

<i>key</i>	
------------	--

Returns

MapNode*

6.6.2.2 getFirst()

```
MapNode * Map::getFirst ( )
```

Returns

MapNode*

6.6.2.3 getNext()

```
MapNode * Map::getNext (
    MapNode * node )
```

Parameters

<i>node</i>	
-------------	--

Returns

MapNode*

6.6.2.4 insert()

```
void Map::insert (
    Pointer key,
    Pointer value )
```

Add a new pair (key --> value) in the map.

Parameters

<i>key</i>	
<i>value</i>	

6.6.2.5 rehash()

```
void Map::rehash ( )
```

Extend the hash table in case the load factor exceeds the predefined threshold.

Parameters

<i>map</i>	The map whose hash table needs to be rehashed
------------	---

6.6.2.6 remove()

```
bool Map::remove (
    Pointer key )
```

Remove from the map the pair (key --> value) that corresponds to the given key.

Returns

true he node with the given key was removed successfully
false he node with the given key was not removed

The documentation for this class was generated from the following files:

- [ADTMap.hpp](#)
- [ADTMap.cpp](#)

6.7 MapNode Class Reference

Public Member Functions

- **MapNode** (Pointer key, Pointer value, State state)
- void **setKey** (Pointer key)
- void **setValue** (Pointer value)
- void **setState** (State state)
- Pointer **getKey** ()
- Pointer **getValue** ()
- State **getState** ()

The documentation for this class was generated from the following file:

- [ADTMap.hpp](#)

6.8 PQueue Class Reference

ADT Priority Queue.

```
#include <ADTPQueue.hpp>
```

Public Member Functions

- **PQueue** (CompareFunc compare, DestroyFunc destroyValue, **Vector** *values)
Construct a new PQueue object.
- **~PQueue** ()
Destroy the PQueue object.
- int **getSize** ()
Get the size of the priority queue.
- Pointer **getMax** ()
Get the max element of the queue.
- Pointer **getMin** ()
Get the min element of the queue.
- void **insert** (Pointer value)
Insert a new element to the queue.
- void **removeMax** ()
Remove the max of the queue.
- DestroyFunc **setDestroyValue** (DestroyFunc destroyValue)
Set the Destroy Value object.
- void **remove** (Pointer value, CompareFunc compare)
Finds (using the given compare function) and removes the node with the given value.
- int **find** (Pointer value, CompareFunc compare)
Finds the given value in the priority queue and returns its nodeId.
- Pointer **nodeValue** (int nodeId)
- void **nodeSwap** (int nodeId1, int nodeId2)
- void **bubbleUp** (int nodeId)
- void **bubbleDown** (int nodeId)
- void **naiveHeapify** (**Vector** *values)

6.8.1 Detailed Description

ADT Priority Queue.

6.8.2 Constructor & Destructor Documentation

6.8.2.1 PQueue()

```
PQueue::PQueue (
    CompareFunc compare,
    DestroyFunc destroyValue,
    Vector * values )
```

Construct a new [PQueue](#) object.

Uses the `compare` function to compare its elements.

If `destroyValue != nullptr`, the `destroyValue` function is called upon removal of an element.

If `values != nullptr`, then the priority queue will be initialized with the values of the vector `values`.

Parameters

<i>compare</i>	
<i>destroyValue</i>	
<i>values</i>	

6.8.2.2 ~PQueue()

```
PQueue::~~PQueue ( )
```

Destroy the [PQueue](#) object.

6.8.3 Member Function Documentation

6.8.3.1 find()

```
int PQueue::find (
    Pointer value,
    CompareFunc compare )
```

Finds the given value in the priority queue and returns its nodeId.

Parameters

<i>value</i>	The value we want to find.
<i>compare</i>	Compare function to use.

Returns

int

6.8.3.2 getMax()

```
Pointer PQueue::getMax ( )
```

Get the max element of the queue.

Returns

Pointer A generic pointer to the max element.

6.8.3.3 getMin()

```
Pointer PQueue::getMin ( )
```

Get the min element of the queue.

Returns

Pointer A generic pointer to the min element.

6.8.3.4 getSize()

```
int PQueue::getSize ( )
```

Get the size of the priority queue.

Returns

int

6.8.3.5 insert()

```
void PQueue::insert (
    Pointer value )
```

Insert a new element to the queue.

Parameters

<i>value</i>	The value of the new element to be inserted.
--------------	--

6.8.3.6 remove()

```
void PQueue::remove (
    Pointer value,
    CompareFunc compare )
```

Finds (using the given compare function) and removes the node with the given value.

Parameters

<i>value</i>	The value we want to remove.
<i>compare</i>	Compare function to use for finding the node we want to remove.

6.8.3.7 removeMax()

```
void PQueue::removeMax ( )
```

Remove the max of the queue.

6.8.3.8 setDestroyValue()

```
DestroyFunc PQueue::setDestroyValue (
    DestroyFunc destroyValue )
```

Set the Destroy Value object.

Parameters

<i>destroyValue</i>	
---------------------	--

Returns

DestroyFunc

The documentation for this class was generated from the following files:

- [ADTPQueue.hpp](#)
- [ADTPQueue.cpp](#)

6.9 Vector Class Reference

ADT [Vector](#).

```
#include <ADTVector.hpp>
```

Public Member Functions

- [Vector](#) (int size, DestroyFunc destroyValue)
Construct a new [Vector](#) object.
- [~Vector](#) ()
Destroy the [Vector](#) object.
- int [getSize](#) ()
Get the current size of the [Vector](#).
- void [insertLast](#) (Pointer value)
Inserts a new element at the end of the vector.
- int [removeLast](#) ()
Removes the last element of the vector.
- Pointer [getAt](#) (int pos)
Get the value at the specified position of the vector.
- int [setAt](#) (int pos, Pointer value)
Set the value at the specified position of the vector.
- Pointer [find](#) (Pointer value, CompareFunc compare)
Find the first element with value equal to value.
- int [findPos](#) (Pointer value, CompareFunc compare)
Find the first element with value equal to value and return its pos.
- DestroyFunc [setDestroyValue](#) (DestroyFunc destroyValue)
Set the Destroy Value.
- [vectorNode](#) * [first](#) ()
Get the first node of the vector.
- [vectorNode](#) * [last](#) ()
Get the last node of the vector.
- [vectorNode](#) * [next](#) ([vectorNode](#) *node)
Get the next node of the vector, after the one given.
- [vectorNode](#) * [previous](#) ([vectorNode](#) *node)
Get the previous node of the vector, before the one given.
- Pointer [nodeValue](#) ([vectorNode](#) *node)
Get the value of the node.
- [vectorNode](#) * [findNode](#) (Pointer value, CompareFunc compare)
Find the first node with value equal to value.

6.9.1 Detailed Description

ADT [Vector](#).

An ADT [Vector](#) using a dynamic array for smart resizing.

6.9.2 Constructor & Destructor Documentation

6.9.2.1 Vector()

```
Vector::Vector (
    int size,
    DestroyFunc destroyValue )
```

Construct a new [Vector](#) object.

The newly created [Vector](#) will be of size size and its elements will be initialized as nullptr.

If destroyValue is not nullptr, it will be called on each element when it is removed from the [Vector](#).

Parameters

<i>size</i>	The initial size of the Vector .
<i>destroyValue</i>	The function to be called on each element when it is removed from the Vector .

6.9.2.2 ~Vector()

```
Vector::~~Vector ( )
```

Destroy the [Vector](#) object.

Deletes all allocated memory of the [Vector](#).

6.9.3 Member Function Documentation**6.9.3.1 find()**

```
Pointer Vector::find (
    Pointer value,
    CompareFunc compare )
```

Find the first element with value equal to value.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

Pointer A pointer to the found value.

6.9.3.2 findNode()

```
vectorNode * Vector::findNode (
    Pointer value,
    CompareFunc compare )
```

Find the first node with value equal to value.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

`vectorNode` The resulting node.

6.9.3.3 findPos()

```
int Vector::findPos (
    Pointer value,
    CompareFunc compare )
```

Find the first element with value equal to value and return its pos.

Parameters

<i>value</i>	The value to look for.
<i>compare</i>	The function to be used for comparison.

Returns

`int` The position of the element.

6.9.3.4 first()

```
vectorNode * Vector::first ( )
```

Get the first node of the vector.

Returns

`vectorNode`

6.9.3.5 getAt()

```
Pointer Vector::getAt (
    int pos )
```

Get the value at the specified position of the vector.

Parameters

<i>pos</i>	The position to look for.
------------	---------------------------

Returns

`Pointer` A pointer to the value at the specified position.

6.9.3.6 getSize()

```
int Vector::getSize ( )
```

Get the current size of the [Vector](#).

Returns

int The size of the [Vector](#).

6.9.3.7 insertLast()

```
void Vector::insertLast (
    Pointer value )
```

Inserts a new element at the end of the vector.

Parameters

<i>value</i>	The value of the new element.
--------------	-------------------------------

If we are at the last element, we create a new array double the size.

6.9.3.8 last()

```
vectorNode * Vector::last ( )
```

Get the last node of the vector.

Returns

[vectorNode](#)

6.9.3.9 next()

```
vectorNode * Vector::next (
    vectorNode * node )
```

Get the next node of the vector, after the one given.

Parameters

<i>node</i>	The node to get the next of.
-------------	------------------------------

Returns

[vectorNode](#)

6.9.3.10 nodeValue()

```
Pointer Vector::nodeValue (
    vectorNode * node )
```

Get the value of the node.

Parameters

<i>node</i>	
-------------	--

Returns

Pointer

6.9.3.11 previous()

```
vectorNode * Vector::previous (
    vectorNode * node )
```

Get the previous node of the vector, before the one given.

Parameters

<i>node</i>	The node to get the previous of.
-------------	----------------------------------

Returns

vectorNode

6.9.3.12 removeLast()

```
int Vector::removeLast ( )
```

Removes the last element of the vector.

Returns

int -1 if the function fails, else 0.

6.9.3.13 setAt()

```
int Vector::setAt (
    int pos,
    Pointer value )
```

Set the value at the specified position of the vector.

Parameters

<i>pos</i>	The position to edit.
<i>value</i>	The new value.

Returns

int -1 if the function fails, else 0.

6.9.3.14 setDestroyValue()

```
DestroyFunc Vector::setDestroyValue (
    DestroyFunc destroyValue )
```

Set the Destroy Value.

Parameters

<i>destroyValue</i>	The new destroy value function.
---------------------	---------------------------------

Returns

DestroyFunc The old destroy value function.

The documentation for this class was generated from the following files:

- [ADTVector.hpp](#)
- [ADTVector.cpp](#)

6.10 vectorNode Class Reference

Class for the vector Node object.

```
#include <ADTVector.hpp>
```

Public Member Functions

- [vectorNode](#) ()
Construct a new vector Node object.
- **vectorNode** (Pointer value)
- Pointer [getValue](#) () const
Get the Value object.
- void [setValue](#) (Pointer value)
Set the Value object.

6.10.1 Detailed Description

Class for the vector Node object.

A vector node is described only its value.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 vectorNode()

```
vectorNode::vectorNode ( ) [inline]
```

Construct a new vector Node object.

Parameters

<i>value</i>	A Pointer to the value.
--------------	-------------------------

6.10.3 Member Function Documentation

6.10.3.1 getValue()

```
Pointer vectorNode::getValue ( ) const [inline]
```

Get the Value object.

Returns

Pointer

6.10.3.2 setValue()

```
void vectorNode::setValue (
    Pointer value ) [inline]
```

Set the Value object.

Parameters

<i>value</i>	
--------------	--

The documentation for this class was generated from the following file:

- [ADTVector.hpp](#)

Chapter 7

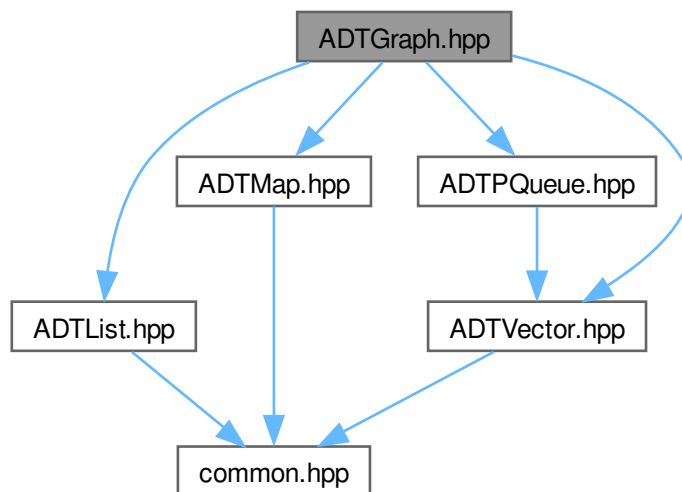
File Documentation

7.1 ADTGraph.hpp File Reference

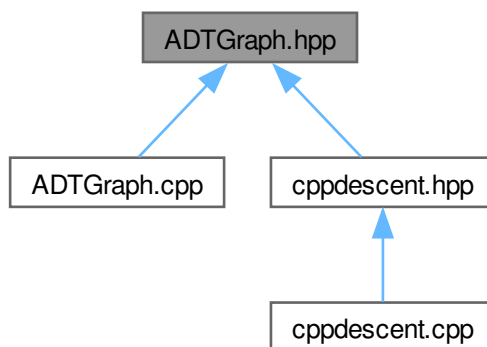
Abstract undirected graph with weighted edges.

```
#include "ADTList.hpp"  
#include "ADTMap.hpp"  
#include "ADTPQueue.hpp"  
#include "ADTVector.hpp"
```

Include dependency graph for ADTGraph.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [Graph](#)
- class [GraphVertex](#)
- class [GraphVertexPair](#)

7.1.1 Detailed Description

Abstract undirected graph with weighted edges.

Author

Phaedon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

7.2 ADTGraph.hpp

[Go to the documentation of this file.](#)

```

00001
00012 #pragma once
00013
00014 #include "ADTList.hpp"
00015 #include "ADTMap.hpp"
00016 #include "ADTPQueue.hpp"
00017 #include "ADTVector.hpp"
00018
00019 class Graph {
00020 private:
00021     Vector* vec;
00022     Map* map;
00023     int size;
00024     CompareFunc compare_vertices;
00025     CompareFunc compare_data;
00026     DestroyFunc destroy;
00027     DestroyFunc destroy_data;
00028     HashFunc hash;
00029
00030 public:
00031     Graph(CompareFunc compare,
00032           DestroyFunc destroy,
00033           DestroyFunc vecDestroy = nullptr);
00034     ~Graph();
00035     int getSize();
00036     void insertVertex(Pointer vertex);
00037     List* getVertices();
00038     void removeVertex(Pointer vertex);
00039     void insertEdge(Pointer vertex1, Pointer vertex2, float weight);
00040     void removeEdge(Pointer vertex1, Pointer vertex2);
00041     float getWeight(Pointer vertex1, Pointer vertex2);
00042     List* getAdjacent(Pointer vertex);
00043     List* getAdjacentVertices(Pointer vertex);
00044     PQueue* getAdjacentPQ(Pointer vertex);
00045     List* getReverseAdjacent(Pointer vertex);
00046     List* getReverseAdjacentVertices(Pointer vertex);
00047     PQueue* getReverseAdjacentPQ(Pointer vertex);
00048     List* getGeneralNeighbors(Pointer vertex);
00049     List* getGeneralNeighborsVertices(Pointer vertex);
00050     PQueue* getGeneralNeighborsPQ(Pointer vertex);
00051     bool isNeighbor(Pointer v1, Pointer v2);
00052     // Map* shortestPathLengths();
00053     void setHashFunction(HashFunc hash);
00054     CompareFunc getCompareData() { return this->compare_data; };
00055     CompareFunc getCompareVertices() { return this->compare_vertices; };
00056     DestroyFunc getDestroy() { return this->destroy; };
00057     DestroyFunc getDestroyData() { return this->destroy_data; };
00058     HashFunc getHash() { return this->hash; };
00059     Vector* getVec() { return this->vec; };
00060     Map* getMap() { return this->map; };
00061 };
00062
00063 class GraphVertex {
00064 private:
00065     Pointer data;
00066     PQueue* neighbors;
00067     PQueue* reverse;
00068     Graph* owner;
00069     bool hasBeenChecked;
00070
00071 public:
00072     GraphVertex(Pointer data, Graph* owner);
00073     ~GraphVertex();
00074     void addNeighbor(Pointer neighbor) { this->neighbors->insert(neighbor); };
00075     void addReverse(Pointer reverse) { this->reverse->insert(reverse); };
00076     void removeNeighbor(Pointer neighbor, CompareFunc compare) {
00077         this->neighbors->remove(neighbor, compare);
00078     };
00079     void removeReverse(Pointer reverse, CompareFunc compare) {
00080         this->reverse->remove(reverse, compare);
00081     };
00082     PQueue* getNeighbors() { return this->neighbors; };
00083     PQueue* getReverse() { return this->reverse; };
00084     Pointer getData() { return this->data; };
00085     Graph* getOwner() { return this->owner; };
00086     void check() { this->hasBeenChecked = true; };
00087     bool checked() { return this->hasBeenChecked; };
00088 };
00089
00090 class GraphVertexPair {
00091 public:
00092     GraphVertexPair(Graph* owner, Pointer vertex1, Pointer vertex2)

```

```

00093         : vertex1(vertex1), vertex2(vertex2), owner(owner){};
00094     Pointer getVertex1() { return this->vertex1; };
00095     Pointer getVertex2() { return this->vertex2; };
00096     Graph* getOwner() { return this->owner; };
00097
00098     private:
00099     Pointer vertex1;
00100     Pointer vertex2;
00101     Graph* owner;
00102 };

```

7.3 ADTList.hpp

```

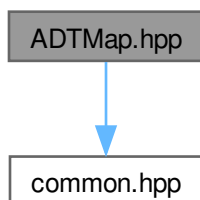
00001
00011 #pragma once
00012 #include "common.hpp"
00013
00014 #define LIST_BOF (ListNode*)0
00015 #define LIST_EOF (ListNode*)0
00016
00024 class ListNode {
00025 public:
00030     ListNode() : next(nullptr), value(nullptr){};
00036     ListNode(Pointer value) : next(nullptr), value(value){};
00042     void setNext(ListNode* next);
00048     ListNode* getNext();
00054     Pointer getValue();
00055
00056 private:
00057     ListNode* next;
00058     Pointer value;
00059 };
00060
00068 class List {
00069 public:
00076     List(DestroyFunc value = nullptr);
00081     ~List();
00088     void insertNext(ListNode* node, Pointer value);
00094     void removeNext(ListNode* node);
00102     Pointer find(Pointer value, CompareFunc compare);
00109     DestroyFunc setDestroyValue(DestroyFunc value);
00118     int getSize();
00119     int increaseSize();
00120     int decreaseSize();
00126     ListNode* getHead();
00132     ListNode* getTail();
00139     ListNode* next(ListNode* node);
00146     Pointer nodeValue(ListNode* node);
00154     ListNode* findNode(Pointer value, CompareFunc compare);
00161     int mergeLists(List* list);
00162
00163 private:
00164     DestroyFunc destroyValue;
00165     int size;
00166     ListNode* head;
00167     ListNode* tail;
00168 };

```

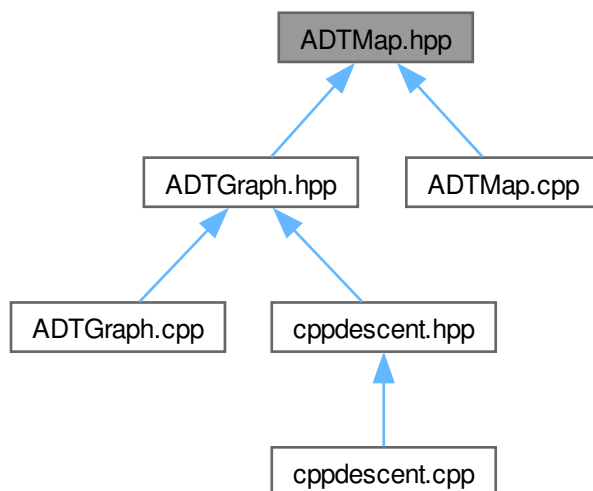

7.4 ADTMap.hpp File Reference

```
#include "common.hpp"
```

Include dependency graph for ADTMap.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [MapNode](#)
- class [Map](#)

Macros

- #define **MAX_LOAD_FACTOR** 0.5
- #define **MAP_EOF** ([MapNode](#)*)0

Enumerations

- enum **State** { **EMPTY** , **OCCUPIED** , **DELETED** }

7.4.1 Detailed Description

Author

Phaedon Seitaniadis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

7.5 ADTMap.hpp

[Go to the documentation of this file.](#)

```
00001
00011 #pragma once
00012
00013 #include "common.hpp"
00014
00015 #define MAX_LOAD_FACTOR 0.5
00016 #define MAP_EOF (MapNode*)0
00017
00018 typedef enum { EMPTY, OCCUPIED, DELETED } State;
00019
00020 class MapNode {
00021 public:
00022     MapNode() : key(nullptr), value(nullptr), state(EMPTY){};
00023     MapNode(Pointer key, Pointer value, State state)
00024         : key(key), value(value), state(state){};
00025     ~MapNode(){};
00026     void setKey(Pointer key) { this->key = key; };
00027     void setValue(Pointer value) { this->value = value; };
00028     void setState(State state) { this->state = state; };
00029     Pointer getKey() { return this->key; };
00030     Pointer getValue() { return this->value; };
00031     State getState() { return this->state; };
00032
00033 private:
00034     Pointer key;
00035     Pointer value;
00036     State state;
00037 };
00038
00039 class Map {
00040 public:
00041     Map();
00042     Map(CompareFunc compare, DestroyFunc destroyKey, DestroyFunc destroyValue);
00043     ~Map();
00044     void insert(Pointer key, Pointer value);
00045     bool remove(Pointer key);
00046     Pointer find(Pointer key);
00047     void rehash();
00048     void setCapacity(int capacity) { this->capacity = capacity; };
00049     void setDestroyKey(DestroyFunc destroyKey) { this->destroyKey = destroyKey; };
00050     void setDestroyValue(DestroyFunc destroyValue) {
```

```

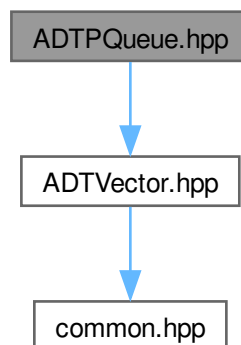
00051     this->destroyValue = destroyValue;
00052 };
00053 void setHashFunction(HashFunc hash) { this->hash = hash; };
00054 MapNode** getArray() { return this->array; };
00055 int getCapacity() { return this->capacity; };
00056 int getSize() { return this->size; };
00057 int getDeleted() { return this->deleted; };
00058 DestroyFunc getDestroyKey() { return this->destroyKey; };
00059 DestroyFunc getDestroyValue() { return this->destroyValue; };
00060 HashFunc getHashFunction() { return this->hash; };
00061 MapNode* getFirst();
00062 MapNode* getNext(MapNode* node);
00063 MapNode* findNode(Pointer key);
00064
00065 private:
00066     MapNode** array;
00067     CompareFunc compare;
00068     DestroyFunc destroyKey;
00069     DestroyFunc destroyValue;
00070     HashFunc hash;
00071     int capacity;
00072     int size;
00073     int deleted;
00074 };

```

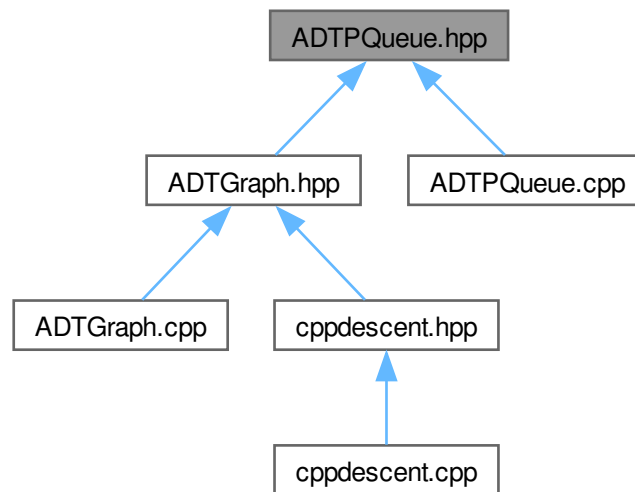
7.6 ADTPQueue.hpp File Reference

#include "ADTVector.hpp"

Include dependency graph for ADTPQueue.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [PQueue](#)
ADT Priority Queue.

7.6.1 Detailed Description

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

Copyright (c) 2023

7.7 ADTPQueue.hpp

[Go to the documentation of this file.](#)

```

00001
00011 #pragma once
00012
00013 #include "ADTVector.hpp"
00014
00019 class PQueue {
00020 public:
00036     PQueue(CompareFunc compare, DestroyFunc destroyValue, Vector* values);
00041     ~PQueue();
00047     int getSize();
00053     Pointer getMax();
00059     Pointer getMin();
00065     void insert(Pointer value);
00070     void removeMax();
00077     DestroyFunc setDestroyValue(DestroyFunc destroyValue);
00086     void remove(Pointer value, CompareFunc compare);
00094     int find(Pointer value, CompareFunc compare);
00095
00096     // Helper functions
00097     // These are used because the node IDs are 1-based, where as the
00098     // vector is 0-based.
00099     Pointer nodeValue(int nodeId);
00100     void nodeSwap(int nodeId1, int nodeId2);
00101     void bubbleUp(int nodeId);
00102     void bubbleDown(int nodeId);
00103     void naiveHeapify(Vector* values);
00104
00105 private:
00106     Vector* vector;
00107     CompareFunc compare;
00108     DestroyFunc destroyValue;
00109 };

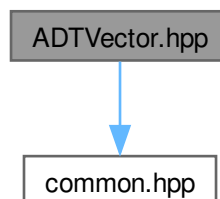
```

7.8 ADTVector.hpp File Reference

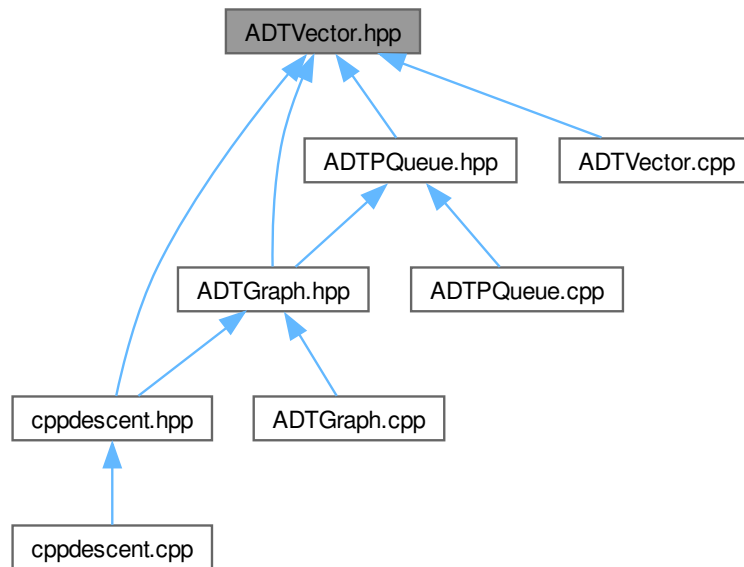
ADTVector implementation using a dynamic array.

```
#include "common.hpp"
```

Include dependency graph for ADTVector.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class [vectorNode](#)
Class for the vector Node object.
- class [Vector](#)
ADT [Vector](#).

Macros

- #define **VECTOR_MIN_CAPACITY** 10
- #define **VECTOR_BOF** ([vectorNode*](#))0
- #define **VECTOR_EOF** ([vectorNode*](#))0

7.8.1 Detailed Description

ADTVector implementation using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

7.9 ADTVector.hpp

[Go to the documentation of this file.](#)

```

00001
00011 #pragma once
00012
00013 #include "common.hpp"
00014
00015 #define VECTOR_MIN_CAPACITY 10
00016
00017 #define VECTOR_BOF (vectorNode*)0
00018 #define VECTOR_EOF (vectorNode*)0
00019
00026 class vectorNode {
00027 public:
00033     vectorNode(){};
00034     vectorNode(Pointer value) : value(value){};
00040     Pointer getValue() const { return value; };
00046     void setValue(Pointer value) { this->value = value; };
00047
00048 private:
00049     Pointer value;
00050 };
00051
00058 class Vector {
00059 public:
00073     Vector(int size, DestroyFunc destroyValue);
00080     ~Vector();
00086     int getSize();
00092     void insertLast(Pointer value);
00098     int removeLast();
01005     Pointer getAt(int pos);
01013     int setAt(int pos, Pointer value);
01021     Pointer find(Pointer value, CompareFunc compare);
01029     int findPos(Pointer value, CompareFunc compare);
01036     DestroyFunc setDestroyValue(DestroyFunc destroyValue);
01042     vectorNode* first();
01048     vectorNode* last();
01055     vectorNode* next(vectorNode* node);
01062     vectorNode* previous(vectorNode* node);
01069     Pointer nodeValue(vectorNode* node);
01077     vectorNode* findNode(Pointer value, CompareFunc compare);
01078
01079 private:
01080     vectorNode* array;
01081     int size;
01082     int capacity;
01083     DestroyFunc destroyValue;
01084 };

```

7.10 common.hpp

```

00001 #pragma once
00002
00003 typedef void* Pointer;
00004
00005 typedef int (*CompareFunc)(Pointer a, Pointer b);
00006
00007 typedef void (*DestroyFunc)(Pointer value);
00008
00009 typedef unsigned int (*HashFunc)(Pointer);

```

7.11 cppdescent.hpp

```

00001
00005 #include "ADTGraph.hpp"
00006 #include "ADTVector.hpp"
00007
00008 typedef float (*DistanceFunc)(Pointer a, Pointer b);
00009
00014 namespace cppdescent {
00015 void deleteFloat(Pointer value);
00016
00017 float* createFloat(float value);
00018
00019 float compareFloats(Pointer a, Pointer b);
00020

```

```

00021 int deleteDatapointVectors(Vector* vec);
00022
00023 int compareGraphVertices(Pointer vertex1, Pointer vertex2);
00024
00025 int compareVertices(Pointer first, Pointer second);
00026
00027 // ===== I/O =====
00042 Vector* readBinData(const char* fp, int dimensions);
00043
00062 void writeBinGraph(const char* fp, Graph* graph, int K);
00063
00073 Graph* readBinGraph(const char* fp, int dimensions, DistanceFunc distance);
00074
00075 // ===== Helper Functions =====
00086 float recall(Graph* bfGraph, Graph* nnGraph, int N, int K);
00087
00088 // ===== Metric Functions =====
00097 float euclideanDistance(Pointer a, Pointer b);
00106 float manhattanDistance(Pointer a, Pointer b);
00114 int compareEdgesEuclidean(Pointer first, Pointer second);
00115
00116 int compareVertexPairsEuclidean(Pointer first, Pointer second);
00124 int compareEdgesManhattan(Pointer first, Pointer second);
00125
00126 // ===== KNN computation =====
00142 Graph* KNNBruteForceGraph(Vector* data,
00143                             int K,
00144                             CompareFunc compare,
00145                             DistanceFunc distance);
00161 Graph* NNDescent_KNNGraph(Vector* data,
00162                             int K,
00163                             float delta,
00164                             DistanceFunc distance);
00178 PQueue* NNDescent_Query(Graph* graph,
00179                           int K,
00180                           CompareFunc compare,
00181                           Vector* query);
00182 }; // namespace cppdescent

```

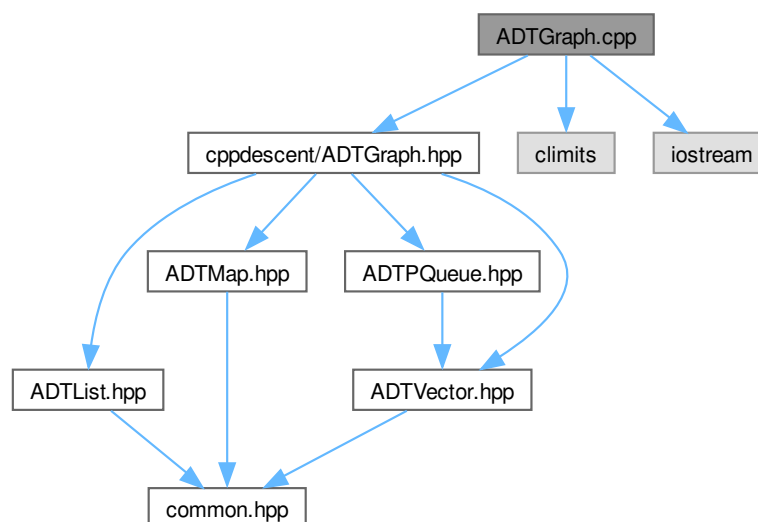
7.12 ADTGraph.cpp File Reference

```

#include "cppdescent/ADTGraph.hpp"
#include <climits>
#include <iostream>

```

Include dependency graph for ADTGraph.cpp:



Functions

- int * **createInt** (int value)
- float * **createFloat** (float value)
- int **compareVertexPair** ([GraphVertexPair](#) *pair1, [GraphVertexPair](#) *pair2)
- int **compareVertices** (Pointer vertex1, Pointer vertex2)
- void **destroyVertexPair** ([GraphVertexPair](#) *pair)
- void **destroyValue** (Pointer value)
- void **destroyVertex** ([GraphVertex](#) *vertex)
- int **compareEdgeWeights** (Pointer a, Pointer b)
- void **destroyEdgePair** ([GraphVertexPair](#) *pair)
- void **swap** (Pointer p, Pointer q)

7.12.1 Detailed Description

Author

Pheadon Seitanidis

Version

0.1

Date

2023-11-01

Copyright

Copyright (c) 2023

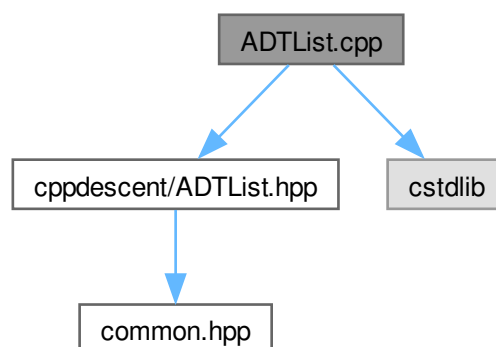
7.13 ADTList.cpp File Reference

Implementation of an ADT linked list.

```
#include "cppdescent/ADTList.hpp"
```

```
#include <cstdlib>
```

Include dependency graph for ADTList.cpp:



7.13.1 Detailed Description

Implementation of an ADT linked list.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-29

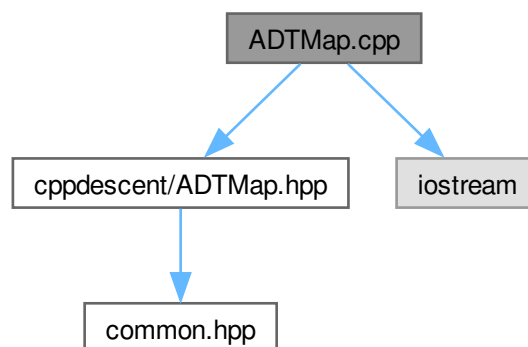
Copyright

Copyright (c) 2023

7.14 ADTMap.cpp File Reference

Implementation of ADTMap.

```
#include "cppdescent/ADTMap.hpp"  
#include <iostream>  
Include dependency graph for ADTMap.cpp:
```

**Variables**

- int `prime_sizes` []

7.14.1 Detailed Description

Implementation of ADTMap.

Author

Phaedon Seitaniadis

Version

0.1

Date

2023-10-29

Copyright

Copyright (c) 2023

7.14.2 Variable Documentation

7.14.2.1 prime_sizes

```
int prime_sizes[]
```

Initial value:

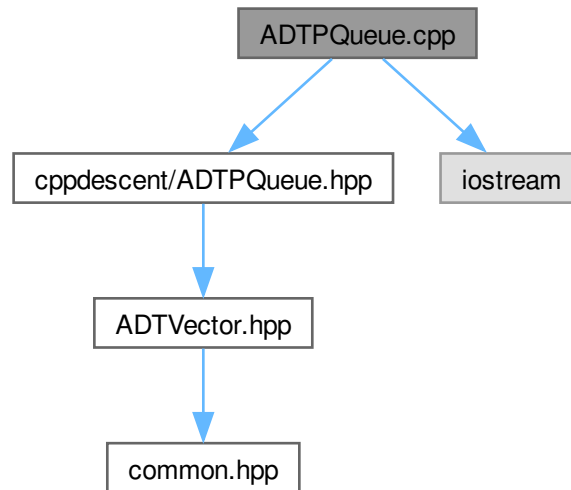
```
= { 53,      97,      193,      389,      769,      1543,      3079,  
    6151,    12289,    24593,    49157,    98317,    196613,    393241,  
    786433,    1572869,    3145739,    6291469,    12582917,    25165843,    50331653,  
    100663319,    201326611,    402653189,    805306457,    1610612741 }
```

We want the capacity of the Hash Table to be a prime number, according to the theory. So the following list consists of prime numbers that are proven good capacity values for a Hash Table. When a rehash is needed, the capacity of the new table will be chosen from this table. If more than 1610612741 elements are needed, we double the capacity of the old table at rehash.

7.15 ADTPQueue.cpp File Reference

An ADT Priority Queue implemented using a heap.

```
#include "cppdescent/ADTPQueue.hpp"  
#include <iostream>  
Include dependency graph for ADTPQueue.cpp:
```



7.15.1 Detailed Description

An ADT Priority Queue implemented using a heap.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

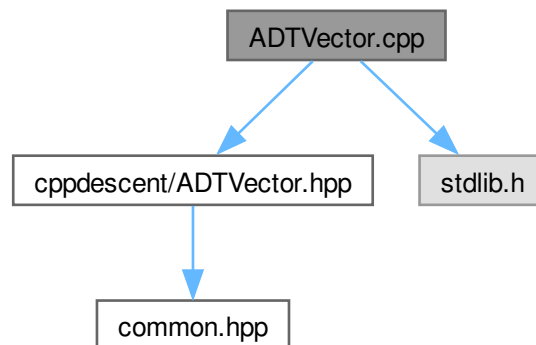
Copyright (c) 2023

7.16 ADTVector.cpp File Reference

Implementation of ADTVector using a dynamic array.

```
#include "cppdescent/ADTVector.hpp"  
#include "stdlib.h"
```

Include dependency graph for ADTVector.cpp:



7.16.1 Detailed Description

Implementation of ADTVector using a dynamic array.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-30

Copyright

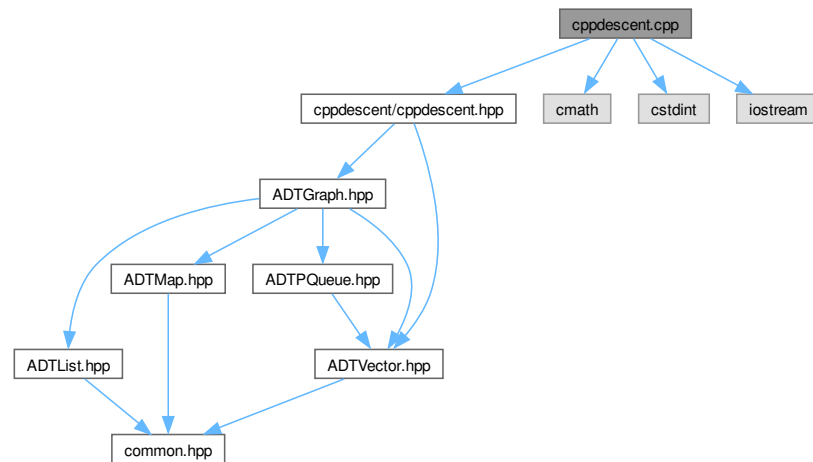
Copyright (c) 2023

7.17 cppdescent.cpp File Reference

Implementation of the cppdescent library.

```
#include "cppdescent/cppdescent.hpp"
#include <cmath>
#include <cstdlib>
#include <iostream>
```

Include dependency graph for cppdescent.cpp:



Functions

- void **swapEdges** ([GraphVertexPair](#) *first, [GraphVertexPair](#) *second)
- void **EdgesBubbleSort** ([GraphVertexPair](#) **edges, int size, CompareFunc compare)
- int **partition** ([GraphVertexPair](#) **edges, int low, int high, CompareFunc compare)
- void **EdgesQuickSort** ([GraphVertexPair](#) **edges, int low, int high, CompareFunc compare)
- bool **EdgesBinarySearch** ([GraphVertexPair](#) **edges, [GraphVertexPair](#) *edge, int low, int high, CompareFunc compare)
- void **destroyEdges** ([GraphVertexPair](#) *pair)
- uint **hashEdge** (Pointer value)
- void **deleteVectors** (Pointer vec)
- [Graph](#) * **sampleGraph** ([Vector](#) *data, int K, CompareFunc compare, DistanceFunc distance)
Creates a random graph, where each vertex has K random neighbors.
- int **updateNN** ([Graph](#) *graph, Pointer v, Pointer u2, float dist, DistanceFunc distance)

7.17.1 Detailed Description

Implementation of the cppdescent library.

Author

Konstantinos Chousos

Version

0.1

Date

2023-10-27

Copyright

Copyright (c) 2023

7.17.2 Function Documentation

7.17.2.1 sampleGraph()

```
Graph * sampleGraph (
    Vector * data,
    int K,
    CompareFunc compare,
    DistanceFunc distance )
```

Creates a random graph, where each vertex has K random neighbors.

The user is responsible for deallocating the graph.

Parameters

<i>data</i>	The Vector with all the points.
<i>K</i>	
<i>compare</i>	
<i>distance</i>	

Returns

Graph* The created graph.

